

# Practica - 20 Cubos Multidimensionales

Luis Gálvez

## Tabla de contenidos

|                                                                       |          |
|-----------------------------------------------------------------------|----------|
| <b>1 Evaluación</b>                                                   | <b>1</b> |
| 1.1 Ejercicio 1: Adquisición de datos climáticos (Copernicus CDS)     | 1        |
| 1.1.1 Desarrollo                                                      | 2        |
| 1.2 Ejercicio 2: Procesamiento del cubo NetCDF (El pipeline perezoso) | 4        |
| 1.2.1 Desarrollo                                                      | 5        |
| 1.3 Preguntas                                                         | 8        |
| 1.3.1 Desarrollo                                                      | 8        |

## 1 Evaluación

### 1.1 Ejercicio 1: Adquisición de datos climáticos (Copernicus CDS)

**Contexto:** Los repositorios climáticos globales contienen petabytes de información. Para su proyecto final (o para este ejercicio), necesitarán descargar un subconjunto de datos de reanálisis ERA5 correspondiente a una región y periodo de su interés.

#### Instrucciones:

1. **Registro:** Diríjanse al portal de *Copernicus Climate Data Store* (CDS) y regístrense para obtener una cuenta gratuita.
2. **Exploración:** Naveguen hacia uno de los siguientes conjuntos de datos:
  - [ERA5 hourly data on single levels from 1940 to present](#)
  - [ERA5-Land hourly data from 1950 to present](#)
  - Otro dataset de Copernicus que sea de interés para su proyecto final.
3. **Selección mediante Interfaz Gráfica:** Utilicen el formulario web para seleccionar:
  - Una variable continua (ej. *2m temperature* o *Total precipitation*).
  - Un año, un mes y todos los días/horas de ese mes.
  - **Importante:** En la sección “Geographical area”, seleccionen “Sub-region extraction” e introduzcan las coordenadas de un *bounding box* de su elección (ej. el departamento o cuenca de su proyecto)

de investigación). Si descargan el mundo entero, el archivo será demasiado pesado.

- Formato: NetCDF.

4. **Automatización (Requisito):** En lugar de hacer clic en “Submit Form”, hagan clic en el botón “**Show API request**”. Esto generará un bloque de código en Python. Utilizando la librería `cdsapi`, integren este código en un script (`.py`) para realizar la descarga automáticamente desde su entorno de programación. Deben instalar la librería `cdsapi`.
5. **Descarga:** Obtengan el archivo `.nc` (asegúrense de que no supere los 500 MB para no saturar sus equipos locales) y guárdenlo en su carpeta `./data_heavy/`.

### 1.1.1 Desarrollo

```
# DESCARGA AUTOMÁTICA ERA5 - PRECIPITACIÓN

# INSTALACIÓN DE LIBRERÍA
# -----
# Ejecutar una sola vez en terminal:
#
# pip install cdsapi
#

# -----
# LIBRERÍAS

import cdsapi
import os

# -----
# CREAR DIRECTORIO DE SALIDA

os.makedirs("./data_heavy", exist_ok=True)

# -----
# CONFIGURACIÓN DE CREDENCIALES CDS
credenciales = """
url: https://cds.climate.copernicus.eu/api
key: 6199561d-6712-45f5-9d32-2aa7fc7b8239
"""

# Crear automáticamente el archivo .cdsapirc
with open("/root/.cdsapirc", "w") as archivo:
    archivo.write(credenciales)
```

```

print(".cdsapirc creado correctamente")

# -----
# CONFIGURAR CLIENTE CDS
client = cdsapi.Client()

# -----
# DATASET ERA5
dataset = "reanalysis-era5-single-levels"

# -----
# SOLICITUD DE DATOS

request = {

    "product_type": ["reanalysis"],

    "variable": ["total_precipitation"],

    "year": ["2005"],

    "month": ["07"],

    "day": [
        "01", "02", "03",
        "04", "05", "06",
        "07"
    ],

    "time": [
        "00:00", "01:00", "02:00",
        "03:00", "04:00", "05:00",
        "06:00", "07:00", "08:00",
        "09:00", "10:00", "11:00",
        "12:00", "13:00", "14:00",
        "15:00", "16:00", "17:00",
        "18:00", "19:00", "20:00",
        "21:00", "22:00", "23:00"
    ],

    "data_format": "netcdf",

    "download_format": "unarchived",

    # Norte, Oeste, Sur, Este

```

```

    "area": [
        5.25,
        -74.25,
        5.00,
        -74.00
    ]
}

# -----
# ARCHIVO DE SALIDA
archivo_salida = "./data_heavy/era5_precipitacion_pacho.nc"

# -----
# DESCARGA AUTOMÁTICA

print("Iniciando descarga ERA5...")

client.retrieve(
    dataset,
    request,
    archivo_salida
)

# -----
# MENSAJE FINAL

print("=====")
print("DESCARGA COMPLETADA")
print("=====")
print(f"Archivo guardado en: {archivo_salida}")

```

## 1.2 Ejercicio 2: Procesamiento del cubo NetCDF (El pipeline perezoso)

**Contexto:** Tienen un archivo NetCDF crudo recién descargado de Copernicus. Deben aplicar el flujo de trabajo aprendido para limpiar, transformar y preparar este hipercubo para análisis posteriores o cartografía.

### Instrucciones de código:

1. **Lectura:** Carguen el archivo `.nc` en su entorno utilizando la estrategia de evaluación perezosa (*lazy evaluation*) correspondiente a su lenguaje (`xarray + dask`, `stars` con `proxy=TRUE`, o `Rasters.jl`).
2. **Auditoría:** Impriman en consola un resumen de las dimensiones del cubo, sus variables y su CRS actual.
3. **Álgebra de Mapas:** Apliquen una transformación matemática a la

variable principal utilizando vectorización (sin bucles).

- *Si descendieron Temperatura:* Conviertan de Kelvin a Celsius.
- *Si descendieron Precipitación:* Conviertan los valores de metros a milímetros (multiplicando por 1000).

4. **Remuestreo Temporal:** El archivo descargado está en resolución horaria. Agrupen la dimensión temporal y calculen la **media diaria** (o la suma diaria, si eligieron precipitación).
5. **Máxima mensual y fecha de ocurrencia:** A partir del cubo diario, calculen el valor máximo mensual para cada píxel. Además, generen una segunda variable (atributo) dentro del cubo que almacene la fecha exacta en la que ocurrió dicha máxima. (*Pista conceptual: busquen funciones que devuelvan el índice del valor máximo, como `idxmax` o índices espaciotemporales según el lenguaje elegido*).
6. **Persistencia:** Exporten el resultado final de este procesamiento a un nuevo archivo NetCDF llamado `mi_zona_procesada.nc`.

### 1.2.1 Desarrollo

```
# LIBRERÍAS

import xarray as xr
import dask
import numpy as np

# -----
# RUTA DEL ARCHIVO

ruta_nc = "./data_heavy/era5_precipitacion_pacho.nc"

# -----
# LECTURA DEL NETCDF (LAZY EVALUATION)

ds = xr.open_dataset(
    ruta_nc,
    engine="h5netcdf",
    chunks={}
)

print("=====")
print("DATASET CARGADO")
print("=====")

# -----
# AUDITORÍA DEL HIPERCUBO
```

```

print("\nDIMENSIONES:")
print(ds.dims)

print("\nVARIABLES:")
print(ds.data_vars)

print("\nCRS:")
print(ds.attrs)

print("\nRESUMEN GENERAL:")
print(ds)

# -----
# IDENTIFICAR VARIABLE PRINCIPAL

precip = ds["tp"]

# -----
# ÁLGEBRA DE MAPAS (VECTORIALIZADA)

precip_mm = precip * 1000

# Actualizar atributos
precip_mm.attrs["units"] = "mm"

precip_mm.attrs["long_name"] = (
    "Precipitación total"
)

print("\nConversión realizada: m -> mm")

# -----
# REMUESTREO TEMPORAL

precip_diaria = precip_mm.resample(
    valid_time="1D"
).sum()

print("\nRemuestreo temporal completado")

# -----
# MÁXIMO MENSUAL POR PÍXEL

max_mensual = precip_diaria.resample(
    valid_time="1ME"

```

```

).max()

max_mensual.name = (
    "precipitacion_maxima_mensual"
)

print("\nMáximo mensual calculado")

# -----
# FECHA DE OCURRENCIA DEL MÁXIMO

# idxmax devuelve la fecha exacta donde ocurrió
# el máximo para cada píxel

fecha_max = precip_diaria.resample(
    valid_time="1ME"
).map(
    lambda x: x.idxmax(dim="valid_time")
)

fecha_max.name = "fecha_maxima_precipitacion"

# -----
# LIMPIAR ATRIBUTOS CONFLICTIVOS - Las fechas NO deben heredar unidades de precipitación

fecha_max.attrs = {
    "long_name": "Fecha de ocurrencia de la precipitación máxima mensual"
}

print("\nFecha de ocurrencia calculada")

# -----
# CREAR DATASET FINAL

ds_final = xr.Dataset({

    "precipitacion_maxima_mensual": max_mensual,

    "fecha_maxima_precipitacion": fecha_max

})

# -----
# EXPORTAR NETCDF FINAL

```

```

ruta_salida = (
    "./data_heavy/mi_zona_procesada.nc"
)

ds_final.to_netcdf(
    ruta_salida
)

print("\n=====")
print("PROCESAMIENTO COMPLETADO")
print("=====")
print(f"Archivo exportado en:")
print(ruta_salida)

```

### 1.3 Preguntas

1. **Sobre Evaluación Perezosa (Lazy Evaluation):** Imaginen que olvidaron hacer el remuestreo temporal a medias diarias y deciden reproyectar (hacer *warping*) el cubo horario completo de Copernicus a un sistema local (ej. EPSG:9377) para luego guardarlo. ¿Qué implicaciones tendría esta decisión en la memoria RAM de su computadora y en los tiempos de procesamiento? Justifiquen por qué el *warping* debe ser uno de los últimos pasos del pipeline.
2. **Sobre Convenciones CF y Tiempo:** Basados en su experiencia resolviendo el Ejercicio 2, ¿su lenguaje de programación decodificó automáticamente las fechas del archivo NetCDF al cargarlo, o tuvieron que realizar alguna operación manual para que el eje de tiempo fuera reconocido como fechas reales y no como simples números? Expliquen brevemente.

#### 1.3.1 Desarrollo

1. Si se reproyecta directamente el cubo horario completo de Copernicus antes de reducir su resolución temporal el sistema debe interpolar y transformar espacialmente cada una de las capas horarias del NetCDF, lo que implica un consumo mucho mayor de memoria RAM debido a que el proceso de *warping* requiere leer bloques completos del archivo, recalcular coordenadas y generar nuevos píxeles en el sistema de referencia destino. En cubos climáticos con cientos o miles de timestamps, esto puede saturar fácilmente la memoria disponible y aumentar considerablemente los tiempos de procesamiento y escritura en el disco.

Es por esto que el *warping* debe realizarse al final del pipeline y primero aplicar operaciones que reduzcan el volumen de datos, operaciones como remuestreo temporal, agregaciones estadísticas o recortes espaciales. Al convertir datos



horarios a diarios o mensuales antes de re proyectar, el número de capas disminuye considerablemente y el proceso de transformación espacial se vuelve mucho más eficiente, por lo que aprovecha la evaluación perezosa (lazy evaluation), ya que las operaciones se encadenan sin cargar inmediatamente todo el cubo en RAM.

2. Las fechas del archivo NetCDF fueron decodificadas automáticamente al momento de cargar el dataset, esto ocurrió porque ERA5 sigue las convenciones CF (Climate and Forecast Metadata Conventions), las cuales almacenan metadatos temporales que indican unidades, calendarios y origen temporal. Gracias a ello, xarray reconoció automáticamente la dimensión temporal y la transformó en objetos `datetime64`, permitiendo realizar operaciones como `resample()` o `idxmax()` sin conversiones manuales.

Sin embargo, fue necesario identificar correctamente el nombre de la dimensión temporal, ya que en este caso el archivo utilizaba “`valid_time2`” en lugar del nombre genérico “`time`”. El problema no estuvo en la decodificación de fechas, sino en asumir el nombre de la coordenada temporal. Una vez corregido esto, las funciones temporales de xarray operaron correctamente sobre fechas reales y no sobre índices numéricos.